Proyecto de Realidad Aumentada

Detección de manos con la cámara usando Ionic, Angular y MediaPipe

Descripción: la aplicación abre la cámara del dispositivo, detecta las manos en tiempo real con inteligencia artificial (MediaPipe Hands) y dibuja un círculo rojo sobre la punta del dedo índice. Esa superposición de elementos virtuales sobre la imagen real es la base de una experiencia de realidad aumentada.

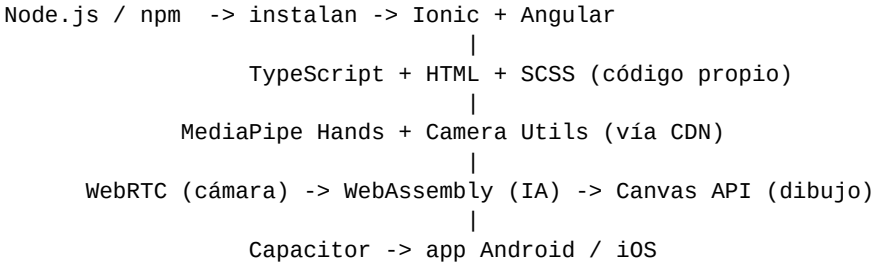
Contenido

1. Tecnologías y librerías utilizadas
2. Estructura del proyecto
3. Flujo general de funcionamiento
4. Paso a paso del código (archivo por archivo)
5. Cómo ejecutar el proyecto
6. Mejoras posibles

1. Tecnologías y librerías utilizadas

Tecnología	Tipo	Para qué se usa en el proyecto
TypeScript	Lenguaje	Escribir la lógica de la aplicación con tipado seguro.
Angular	Framework	Organizar la app en componentes, conectar HTML con lógica y controlar el ciclo de vida de la pantalla.
Ionic	Framework UI	Crear la app móvil/web con un solo código y ejecutar el servidor de desarrollo.
MediaPipe Hands	Librería de IA (Google)	Detectar manos y devolver 21 puntos (landmarks) por mano. Es el núcleo del proyecto.
MediaPipe Camera Utils	Librería	Encender la cámara y entregar cada frame a MediaPipe Hands.
Canvas API (HTML5)	API del navegador	Dibujar el video y los círculos sobre la imagen (efecto AR).
WebRTC / getUserMedia	API del navegador	Acceso a la cámara (la usa Camera Utils internamente). Requiere HTTPS o localhost.
WebAssembly (WASM)	Tecnología del navegador	Ejecutar el modelo de IA a alta velocidad (lo usa MediaPipe internamente).
HTML5	Lenguaje de marcado	Elementos <video> y <canvas> y el index.html donde se cargan las librerías.
SCSS	Preprocesador CSS	Estilos: pantalla completa, ocultar video y efecto espejo.
jsDelivr (CDN)	Servicio de distribución	Cargar los scripts y el modelo de MediaPipe desde internet.
Node.js y npm	Entorno / gestor de paquetes	Instalar Angular, Ionic y dependencias (carpeta node_modules).
Capacitor (opcional)	Herramienta de Ionic	Empaquetar la app web como app nativa Android/iOS con permisos de cámara.
Visual Studio Code	Editor	Entorno donde se desarrolla el proyecto.

Cómo se relacionan



2. Estructura del proyecto

```
camara_ar/
 |-- node_modules/          (dependencias instaladas)
 |-- src/
 |    |-- app/
 |    |    |-- home/
 |    |    |    |-- home.page.html   (video + canvas)
 |    |    |    |-- home.page.scss   (estilos)
 |    |    |    |-- home.page.ts     (lógica de IA y dibujo)
 |    |    |-- app.component.*       (componente raíz)
 |    |    |-- app.routes.ts         (rutas)
 |    |-- assets/
 |    |-- environments/
 |    |-- theme/
 |    |-- global.scss
 |    |-- index.html                (carga MediaPipe)
```

3. Flujo general de funcionamiento

El programa repite este ciclo unas 30 veces por segundo:

```
Cámara -> captura un frame -> MediaPipe detecta la mano -> devuelve 21 puntos -> se dibuja sobre el canvas

1. Angular dibuja el HTML
2. ngAfterViewInit() se ejecuta
3. Se crea Hands y se configura (setOptions)
4. Se registra onResults (qué hacer con cada resultado)
5. hands.initialize() carga el modelo
6. Camera.start() enciende la cámara
7. Por cada frame:
     onFrame -> hands.send(video) -> MediaPipe analiza -> onResults
     -> drawImage (pinta el video) -> arc + fill (pinta el círculo)
8. Al salir de la página: ngOnDestroy() apaga todo
```

4. Paso a paso del código

4.1 index.html: cargar las librerías

```
<script src="https://cdn.jsdelivr.net/npm/@mediapipe/hands@0.4.1675469240/hands.js"
        crossorigin="anonymous"></script>
<script src="https://cdn.jsdelivr.net/npm/@mediapipe/camera_utils@0.3.1675466862/camera_utils.js"
        crossorigin="anonymous"></script>
```

Elemento	Significado
hands.js	Contiene el modelo de IA que detecta manos. Crea la clase global **Hands**.
camera_utils.js	Abre la cámara y entrega frames de forma continua. Crea la clase global **Camera**.
crossorigin="anonymous"	Permite cargar el script desde otro dominio (CDN) sin problemas de seguridad.

Se colocan dentro del <head>, porque MediaPipe funciona como script global del navegador.

4.2 home.page.html: elementos visuales

```
<div class="ar-container">
  <video #videoElement class="input-video" playsinline></video>
  <canvas #canvasElement class="output-canvas"></canvas>
</div>
```

Elemento	Significado
<video>	Recibe la imagen de la cámara. Está oculto; solo sirve como fuente de datos.
<canvas>	Hoja de dibujo donde se pinta el video y los círculos. Es lo que ve el usuario.
#videoElement / #canvasElement	Etiquetas de Angular para acceder a los elementos desde TypeScript.
playsinline	Evita que iOS abra el video en pantalla completa.

4.3 home.page.scss: estilos

```
.ar-container { position: relative; width: 100%; height: 100vh;
                overflow: hidden; background: #000; }
.input-video  { display: none; }
.output-canvas { position: absolute; top: 0; left: 0;
                 width: 100%; height: 100%;
                 object-fit: cover; transform: scaleX(-1); }
```

Regla	Significado
.ar-container	Contenedor de pantalla completa con fondo negro. position: relative permite ubicar el canvas dentro.
.input-video { display: none }	Oculta el video original.
.output-canvas	Con position: absolute y 100% de ancho/alto, el canvas cubre todo el contenedor.

Regla	Significado
transform: scaleX(-1)	Voltea la imagen horizontalmente (efecto espejo, como un selfie).

4.4 home.page.ts: la lógica

Importaciones y variables globales

```
import { Component, ElementRef, ViewChild, AfterViewInit, OnDestroy } from '@angular/core';

declare const Hands: any;
declare const Camera: any;
```

Elemento	Qué es
Component	Decorador que convierte la clase en un componente de Angular.
ViewChild	Permite obtener un elemento del HTML usando su #etiqueta.
ElementRef	Envoltorio de Angular alrededor de un elemento HTML real.
AfterViewInit	Interfaz para ejecutar código cuando el HTML ya se dibujó.
OnDestroy	Interfaz para ejecutar código cuando el componente se cierra.
declare const Hands / Camera	Le dice a TypeScript que estas clases existen (vienen de los scripts del index.html).

Decorador @Component y @ViewChild

```
@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
})
export class HomePage implements AfterViewInit, OnDestroy {
  @ViewChild('videoElement')  videoElement!: ElementRef<HTMLVideoElement>;
  @ViewChild('canvasElement') canvasElement!: ElementRef<HTMLCanvasElement>;
```

@Component conecta la clase con su HTML y su SCSS. **@ViewChild** busca en el HTML el elemento con la etiqueta indicada y lo guarda en una variable. El símbolo **!** indica que Angular asignará el valor antes de usarlo.

ngAfterViewInit(): punto de partida

```
const video  = this.videoElement.nativeElement;
const canvas = this.canvasElement.nativeElement;
const ctx    = canvas.getContext('2d')!;
```

Angular la ejecuta una sola vez, cuando el HTML ya existe. **nativeElement** da acceso al elemento HTML real y **getContext('2d')** devuelve el "pincel" (ctx) con el que se dibuja en el canvas.

Crear y configurar el detector de manos

```
this.hands = new Hands({
  locateFile: (f: string) =>
    `https://cdn.jsdelivr.net/npm/@mediapipe/hands@0.4.1675469240/${f}`,
});

this.hands.setOptions({
  maxNumHands: 2,
  modelComplexity: 1,
  minDetectionConfidence: 0.6,
  minTrackingConfidence: 0.6,
});
```

locateFile le indica a MediaPipe en qué URL descargar sus archivos adicionales (modelo .tflite, archivos .wasm).

Opción	Significado
maxNumHands	Máximo de manos a detectar a la vez.
modelComplexity	0 = modelo ligero y rápido; 1 = más preciso pero más pesado.
minDetectionConfidence	Seguridad mínima (0 a 1) para aceptar que lo detectado es una mano.
minTrackingConfidence	Seguridad mínima para seguir una mano entre frames; si baja, la busca de nuevo.

onResults(): la función más importante

```
this.hands.onResults((r: any) => {
  if (canvas.width !== video.videoWidth) {
    canvas.width  = video.videoWidth;
    canvas.height = video.videoHeight;
  }

  ctx.drawImage(r.image, 0, 0, canvas.width, canvas.height);

  r.multiHandLandmarks?.forEach((mano: any) => {
    const punta = mano[8];
    ctx.beginPath();
    ctx.arc(punta.x * canvas.width, punta.y * canvas.height, 30, 0, 2 * Math.PI);
    ctx.fillStyle = 'rgba(255, 0, 0, 0.8)';
    ctx.fill();
  });
});
```

Es un **callback**: MediaPipe lo llama cada vez que termina de analizar un frame. El parámetro **r** contiene **r.image** (el frame analizado) y **r.multiHandLandmarks** (un arreglo con una entrada por cada mano detectada).

Parte	Qué hace
Ajustar tamaño del canvas	Iguala la resolución del canvas con la del video. Solo se hace si cambia, porque asignar canvas.width borra el contenido.
ctx.drawImage(...)	Pinta el frame de la cámara en el canvas. Parámetros: imagen, X, Y, ancho, alto.
?.forEach(...)	El ?. evita errores si no hay manos; forEach repite el código por cada mano.

Parte	Qué hace
mano[8]	Punto 8 = punta del dedo índice (ver tabla de landmarks).
ctx.beginPath()	Inicia un nuevo trazo.
ctx.arc(x, y, r, 0, 2π)	Dibuja un círculo. Se multiplica x e y por el tamaño del canvas porque vienen normalizadas (0 a 1).
ctx.fillStyle / fill()	Define el color (rojo, 80% opacidad) y rellena la figura.

Los 21 puntos (landmarks) de la mano

Número	Parte de la mano
0	Muñeca
4	Punta del pulgar
8	Punta del índice
12	Punta del dedo medio
16	Punta del anular
20	Punta del meñique

Iniciar el modelo y la cámara

```
await this.hands.initialize();

this.camera = new Camera(video, {
  onFrame: async () => await this.hands.send({ image: video }),
  width: 1080,
  height: 1080,
  facingMode: 'user',
});
await this.camera.start();
```

Elemento	Significado
hands.initialize()	Descarga y prepara el modelo de IA. Es asíncrona, por eso se usa await.
new Camera(video, ...)	Configura la cámara usando el elemento video.
onFrame	Se ejecuta en cada frame nuevo; envía la imagen a MediaPipe con hands.send(), que luego dispara onResults.
width / height	Resolución solicitada a la cámara.
facingMode	'user' = cámara frontal; 'environment' = cámara trasera.
camera.start()	Pide permiso al usuario, enciende la cámara y comienza el bucle de frames.

ngOnDestroy(): limpiar recursos

```
ngOnDestroy(): void {
  this.camera?.stop();
  this.hands?.close();
}
```

Angular la ejecuta al salir de la página. Apaga la cámara y libera la memoria del modelo; de lo contrario la cámara seguiría encendida y consumiendo recursos.

5. Cómo ejecutar el proyecto

Requisitos y creación

```
node -v                      # recomendado Node 18 o superior
npm i -g @ionic/cli
ionic start camara_ar blank --type=angular
cd camara_ar
```

Ejecutar en el navegador

```
ionic serve
```

Abre http://localhost:8100 y acepta el permiso de la cámara.

Probar en el celular

Los navegadores solo permiten la cámara en **HTTPS** o **localhost**. Opciones:

```
# Opción A: HTTPS local
ionic serve --external --ssl

# Opción B: app nativa con Capacitor
ionic build
npm i @capacitor/android
npx cap add android
npx cap sync
npx cap open android
```

En android/app/src/main/AndroidManifest.xml agregar:

```
<uses-permission android:name="android.permission.CAMERA" />
```

6. Mejoras posibles

Mejora	Descripción
Esqueleto de la mano	Usar drawing_utils de MediaPipe (drawConnectors y drawLandmarks).
Detección de gestos	Por ejemplo un pellizco: medir la distancia entre el pulgar (4) y el índice (8).
Objetos virtuales	Dibujar imágenes o emojis sobre la mano con ctx.drawImage.
Objetos 3D	Integrar Three.js sobre el canvas para modelos 3D que sigan la mano.
Interacción	Crear botones virtuales que se activen al tocarlos con el dedo.

Frase para presentar el proyecto

"Aplicación de realidad aumentada desarrollada con Ionic y Angular (TypeScript), que usa MediaPipe Hands para detectar manos en tiempo real mediante la cámara y la Canvas API para superponer

elementos gráficos sobre la imagen."